

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

📅 작성일시	@2025년 3월 13일
📁 강의 번호	BASE
📁 유형	논문리뷰
📎 자료	https://arxiv.org/pdf/1810.04805
☑️ 복습	☐

1. 들어가며

자연어 처리 분야에서 다양한 모델이 제안되어 왔음. 그러나 대부분의 모델들은 특정 작업에 대해 특화된 아키텍처를 필요로 했으며, 이는 모델의 범용성을 제한함. 이러한 문제를 해결하기 위해, BERT가 제안되었음. 이번 글에서는 BERT의 핵심 개념과 구조, 그리고 그 성능과 의의에 대해 깊이 있게 다뤄볼 예정임.

첫 번째 섹션에서는 GPT-1과 같은 기존 모델들의 한계를 살펴볼 예정임. GPT-1의 Unidirectional 학습 방법은 일부 작업에서는 효과적이었지만, 다양한 NLP 작업에서의 범용성을 제한했음. 이러한 한계를 극복하기 위해 BERT는 양방향 컨텍스트를 활용하여 더욱 강력한 표현 학습 능력을 제공함.

다음 섹션에서는 BERT의 두 단계 학습 방법과 아키텍처, 입력/출력 표현에 대해 상세히 설명함. BERT는 pretraining과 fine-tuning 두 단계로 구성되어 있어, 다양한 NLP 작업에 유연하게 적용할 수 있음. 이를 통해 BERT는 다양한 NLP 작업에서 뛰어난 성능을 보이며, 기존 모델들의 한계를 극복함.

세 번째 섹션에서는 BERT의 Pretraining 과정을 자세히 살펴봄. Masked LM과 Next Sentence Prediction 두 가지 방법을 사용하여 모델을 사전 학습함. 이 과정에서 BERT는 대량의 텍스트 데이터에서 풍부한 언어 표현을 학습함.

다음으로, Fine Tuning 과정에 대해 간략히 소개함. 이 과정을 통해 BERT는 특정 작업에 더욱 특화된 성능을 발휘할 수 있음. Fine Tuning을 통해 BERT는 다양한 NLP 작업에서 뛰어난 성능을 보이며, 기존 모델들의 성능을 뛰어넘음.

다섯 번째 섹션에서는 BERT의 실험 결과를 공유함. 여러 NLP 작업에서의 성능 비교와 ablation 실험 결과를 통해 BERT의 효과를 확인할 수 있음. 이를 통해 BERT의 강력한 성능과 범용성을 확인할 수 있음.

여섯 번째 섹션에서는 BERT와 GPT-1의 차이점을 비교 분석함. 아키텍처, 학습 방법, 특화된 기능, 데이터 처리 방향 등 여러 측면에서 두 모델을 비교함. 이를 통해 BERT와 GPT-1의 각각의 장단점을 이해할 수 있음.

다음 섹션에서는 BERT의 장단점에 대해 상세히 분석함. BERT의 강점과 약점을 이해함으로써, 언제 BERT를 사용하는 것이 적절한 한지에 대한 인사이트를 제공함. 이를 통해 BERT의 적절한 활용 방법을 이해할 수 있음.

마지막 섹션에서는 BERT의 의의에 대해 논의함. BERT는 NLP 분야에 어떤 영향을 미쳤는지, 그리고 앞으로 어떻게 발전될 수 있는지에 대해 탐구함. BERT의 영향력과 미래의 발전 가능성을 통해, BERT의 중요성을 이해할 수 있음.

2. 기존 방법의 문제점

이번 챕터에서는 BERT가 풀고자 했던 기존 방법의 문제점을 살펴봄. 이 당시 대표적인 기존 방법이라고 하면 ELMO, GPT-1 등이 있음. 이들의 공통적 한계는 **바로 한방향으로만 학습(Unidirectional)해야 한다는 점**임. GPT-1은 Transformer의 Decoder를 토대로 구성되어 있음. 따라서 Autoregressive한 방식으로 입력으로 순차적으로 받음. 그리고 나서 다음 단어를 예측하는 방법을 통해 학습함.

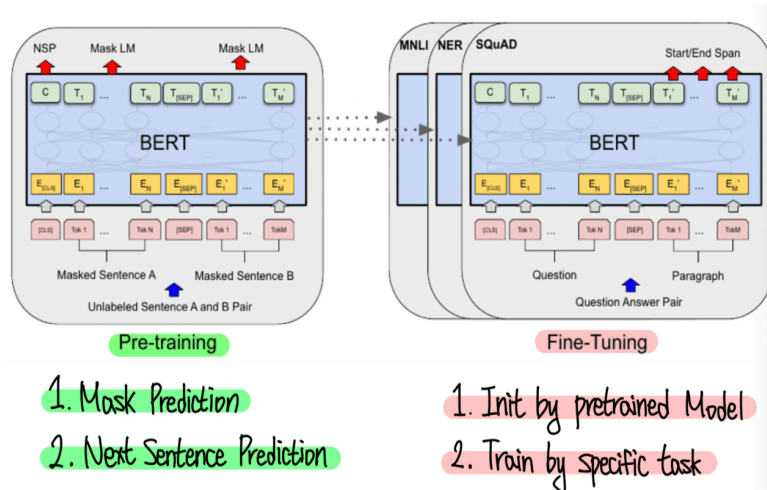
BERT의 저자들은 이러한 한 방향 학습 방법보다, 양방향 학습이 더 유리하다고 주장함. GPT-1과 같이 한 방향으로 순차적으로 입력을 받는 학습 방법도 유리한 분야가 있음. 바로 다음 단어 생성하기 문제임. GPT를 토대로 사람과 대화하는 chatGPT 서비스가 출시된 이유임. 하지만 대부분의 NLP 문제는 이보다 더 긴 Context를 이해하고 대답하는 능력이 필요함. 대표적으로 Question Answering 문제를 생각해봄. 이 문제에서는 문제 상황과 질문을 토대로 대답을 해야하는데, 이를 위해서 **양방향으로 문맥을 추출하는 방법이 더 유리함**.

3. BERT

BERT는 양방향으로 문장을 읽으며 문맥을 이해할 수 있는 모델 개발에 초점을 맞춤.

3-1 전체 구성

BERT도 GPT-1 과 동일하게 먼저 Pretraining을 하고 Fine Tuning하는 구조임.



BERT의 Pretraining 단계에서는 2가지 방법을 사용하여 모델을 학습함. 첫 번째 방법은 입력 문장의 일부 단어를 가린 뒤, 단어를 맞추는 Mask Prediction임. 두 번째 방법은 두 개 문장을 입력으로 받아 두 번째 문장이 첫 번째 문장의 다음 문장인지 맞추는 Next Sentence Prediction임. Pretraining에 대한 내용은 아래에서 자세히 살펴봄.

Pretraining이 완료된 BERT는 일정 부분 언어를 이해하게 됨. 문장의 일부 단어를 맞추고, 두 문장의 관계를 맞추기 위해서는 각 단어의 관계, 문장 구성 등 언어를 일정 부분 이해해야 하기 때문임. 하지만 특정 문제에 특화되어 더욱 성능을 높이기 위해서는 Fine Tuning은 Pretraining된 모델을 기반으로 학습을 시작함. 해당 문제 데이터셋을 사용하여 모델 가중치를 Fine Tuning해주게 됨.

3-2 Architecture

BERT는 Transformer의 Encoder를 기반으로 구성되어 있음.

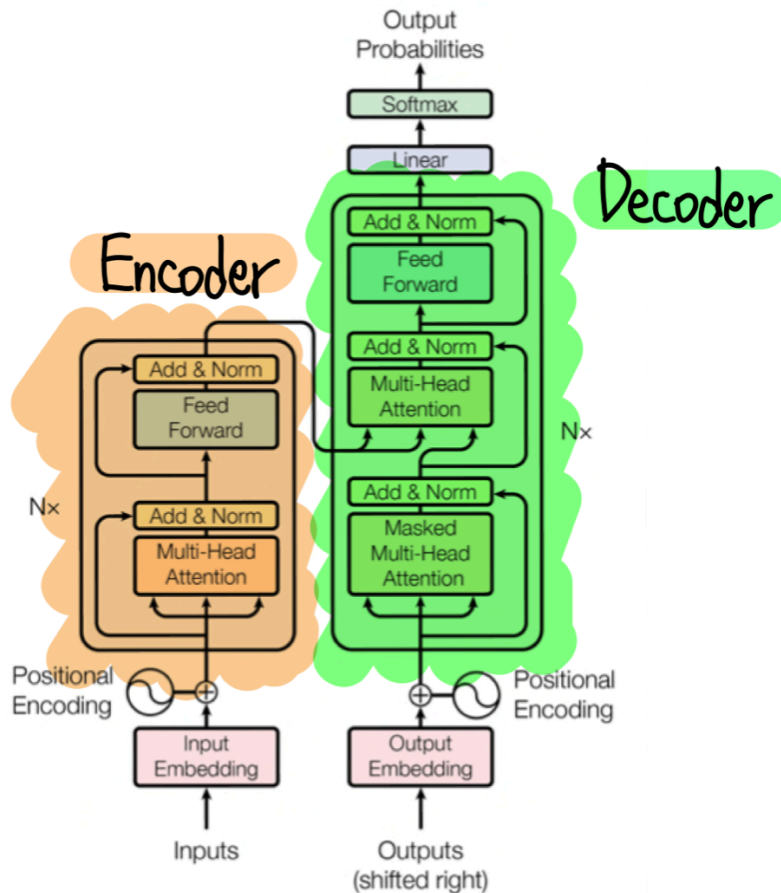


Figure 1: The Transformer - model architecture.

Microsoft E

Transformer의 구조를 표현한 그림임. Decoder의 역할을 번역 문제를 예시로 생각해 보면, Decoder는 번역해야 할 전체 정보와 지금까지 번역한 정보를 입력받음. 그리고 나서 다음으로 번역되어야 하는 단어를 예측함. 이러한 특성으로 Transformer의 Decoder는 데이터를 순차적으로 처리할 수 밖에 없음. 지금까지 번역한 단어들을 순차적으로 업데이트하며 Autoregressive 방식으로 재입력 받으며 번역해야 하기 때문임.

반면 Encoder는 입장이 다름. Transformer에서 Encoder의 역할은 주어진 문장에서 각 단어들 간의 상관관계를 고려하여 데이터를 재구성해주는 것임. 단어를 순차적으로 입력받을 필요가 없음. 문장 번역 문제에서 Encoder는 번역해야 할 문장 전체를 한 번에 입력받음. 이 문장에서 단어들 간 관계를 고려하여 Self Attention 결과에 따라 가중치를 반영해줌. 여기까지가 Encoder의 역할임. 즉, Encoder는 단방향 처리도, 양방향 처리도 가능한 구조임. 이러한 이유로 BERT에서는 Transformer의 Encoder를 기반으로 모델을 구성함.

3-3 Input/Output Representation

BERT는 두 개 문장 관계를 예측하는 방법으로 Pretraining을 한다고 언급함. 두 개 문장을 입력받고 이 전체 문장 덩어리는 어떻게 구성해서 처리할까?

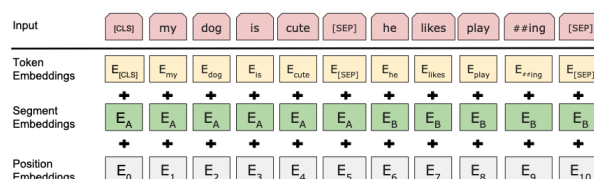


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

입력 단어들은 크게 세 가지 Embedding을 사용해서 최종적으로 BERT가 처리할 수 있는 데이터로 변환됨.

첫 번째는 **Token Embedding**임. 대부분의 LLM에서 공통적으로 사용되는 방법임. 사람이 사용하는 단어를 딥러닝 모델들이 이해할 수 있는 고차원 벡터로 변환하는 역할을 함. BERT에서는 Wordpiece Tokenizer를 사용함. 이때 입력 받은 단어 외에 특수 토큰들이 존재함.

[CLS]는 Classification을 상징하는 토큰임. 최종적으로 Classification을 위해 사용되는 토큰을 말함. 즉 입력 받은 모든 단어들을 대표할 토큰이 되는 것임. 모든 연산을 거쳤을때 문장의 모든 정보는 이 [CLS] 토큰에 압축되고, 따라서 이 [CLS] 토큰만을 사용하여 이후 Task를 수행함. 다음으로 [SEP] 토큰이 있는데 이는 Separation, 즉 문장간 분리를 의미하는 토큰임. 여러 문장을 입력 받았을때, 각 문장을 구분해주는 표현이 있어야 하는데 사람의 언어에서는 구두점(.)에 해당함. 이를 위해 [SEP] 토큰을 사용.

두 번째는 **Segment Embedding** 임. 예를 들어 다음 문장을 예측하는 문제를 생각해 볼 때 2개의 Text를 한번에 입력 받아야 함. 무엇이 첫번째 Text 이고, 무엇이 두번째 Text 인지를 모델에게 알려줘야 하는데 이를 위해 사용하는 것이 Segment Embedding 임. 두 개 문장에 각각 서로 다른 Embedding을 더해주는 것으로 모델은 이를 통해 무엇이 첫 번째 Text이고, 무엇이 두 번째 Text 인지 알 수 있음.

세 번째는 **Positional Embedding** 임. 이는 입력된 데이터에서의 순서를 의미함. Transformer는 입력 데이터의 순서를 고려하지 않으므로, 순서를 구분해 주기 위한 Positional Encoding이 필요함. 이는 Transformer에서와 동일한 방법으로 적용해줌.

4. Pretraining

4-1 Masked LM

첫 번째 pretraining 방법은 Masked LM임. 입력 문장의 일부 단어를 지우고 이렇게 지운 문장을 모델에게 입력해주고 지워진 단어를 예측하도록 학습함. 예를 들어 원래 "I have a pen" 이라는 문장이 있었다고 가정. 이때 Masking을 적용하여 모델은 "I have a [MASK]" 문장을 입력 받는 것이죠. 정확하게는 Masking 처리된 토큰의 마지막 Hidden Vector를 Softmax에 통과시켜 정답 단어를 예측하도록 학습함. 이렇게 전체 문장의 15%를 Masking 하여 학습을 진행.

하지만 이대로 진행하면 문제가 좀 있는데 BERT 입장에서 한번 생각해보면 지금 하고 있는 작업은 Pretraining 임. 말하자면 몸풀기 인거죠. 진짜 잘해야 되는 시험은 이게 아니고요. 진짜로 해야 되는 시험은 문장 분류하기 (Classification) 같은 다양한 NLP 문제임. 그런데 BERT는 [MASK] 토큰에 대해서만 실제 단어가 무엇인지 예측하도록 학습한다? 이 말은 [MASK] 토큰이 아니면 예측할 수 없다는 말과 동일하죠. 이래서는 곤란하잖아요. 그래서 BERT에서는 약간의 트릭을 추가함. 전체 단어의 15%를 Masking 처리한다고 했는데요. 이 15% 중에서 10%는 랜덤 토큰으로 바꿔줌. 예를 들어 아까와 같이 "I have a pen" 문장을 생각해보면 이번에는 pen을 [MASK] 토큰이 아닌 wallet 이라는 단어로 바꿔줌. 그리고는 이 wallet 자리에 원래 어떤 단어가 있어야했는지를 맞추도록 학습하는거임.

두 번째로 15% 중에서 10%는 [MASK] 토큰으로 변환하지 않고 그대로 놔둠. 모델 입장에서는 정답을 보고 정답을 맞추는 셈이 됨. 이러한 방법을 사용해서 BERT는 꼭 [MASK] 토큰에 대해서만 단어를 예측하는게 아님을 알게 됨. 덕분에 어떠한 위치의 단어에 대해서도 문맥을 고려하여 적절한 단어를 예측할 수 있는 능력을 갖게 됨.

4-2 Next Sentence Prediction

두 번째 Pretraining 방법은 Next Sentence Prediction 임. 말 그대로 다음 문장을 예측하는 문제로 사실 다음 문장을 예측해서 생성하는 게 아닌 입력으로 두개의 문장을 받음. 그리고 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지를 맞추는 Binary Classification을 학습하는 것.

이러한 문제를 위한 데이터 구성 방법은 먼저 텍스트 코퍼스 전체에서 문장 쌍을 추출하고 이렇게 추출한 문장 쌍들은 전부 이어진 문장이기에, BERT는 1로 구분하도록 학습해야 하는 데이터임. 이제 이 문장쌍 50%의 두 번째 문장을 임의의 문장으로 바꿔줌. 이렇게 바뀐 50%의 데이터는 다음 문장에 해당 안되서 BERT가 0으로 구분해야 하는 데이터임.

이렇게 BERT는 입력 받은 두개 문장에 대해 Binary Classification을 학습함으로써 문장의 이어짐 여부를 학습할 수 있음. 이러한 Next Sentence Prediction 문제는 Masked LM 문제와 학습 목표가 조금 다른데 Masked LM은 숨겨진 단어를 예측하는 문제라서 모델 입장에서는 단어에 초점을 맞추게 됨. 반면 Next Sentence Prediction 문제는 두 문장의 관계를 파악해야 하는 문제임. 따라서 모델 입장에서는 문장에 초점을 맞춰야 하고, 더욱 넓은 범위의 이해를 요구함. 이렇게 BERT는 상호 보완적인 두 가지 Pretraining 방법을 동시에 사용하여 더욱 다채로운 성능을 갖는 언어 모델이 될 수 있음.

5. Fine Tuning

다음은 이렇게 Pretraining을 완료한 모델이 각 문제에 맞춰 Fine Tuning 하는 방법을 살펴봄. 사실 BERT의 핵심은 Pretraining 이고, Fine Tuning에서는 특별한 주목할 만한 포인트가 없음.

일반적인 Fine Tuning 절차를 따르는데 먼저 Pretraining 완료된 BERT 모델을 로드하고 특정 작업에 맞도록 출력 Layer를 추가 구성해줌. 이제 준비된 특정 작업 데이터로 Fine Tuning 학습을 해줌. 이때 적절한 Learning Rate 등 다양한 하이퍼파라미터 튜닝 작업이 필요함.

6. 실험 결과

6-1 성능 비교

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

첫 번째는 OpenAI GPT와 BERT_{BASE}의 성능 비교입니다. 이 두 모델은 거의 동일한 크기를 갖고 있음. 모든 데이터셋에서 BERT의 성능이 GPT-1 보다 높은 모습을 볼 수 있음. 이를 통해 BERT가 제안한 Encoder 구조에 양방향으로 학습하는 방법과 두 가지의 Pretraining 이 효과적이었음을 알 수 있음.

두 번째는 BERT_{BASE}와 BERT_{LARGE}의 성능 비교임. 모든 데이터셋에서 LARGE 모델의 성능이 BASE 모델보다 좋은데, 이를 통해 BERT는 사이즈가 커질수록 성능이 더 좋아짐을 알 수 있음.

6-2 Pretraining 효과

Tasks	Dev Set				
	MNLI-m (Acc)	QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

Table 5: Ablation over the pre-training tasks using the BERT_{BASE} architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

위 그림은 다양한 Pretraining 방법에 따른 성능을 비교한 표임. 두 번째 행인 No NSP는 Next Sentence Prediction 학습을 하지 않은 모델을 의미함. 세 번째 행의 LTR은 GPT와 같이 한방향 학습 모델을 의미함. 이 표를 통해 두 가지 시사점을 얻을 수 있음.

첫 번째는 Next Sentence Prediction의 효과임. 첫번째, 두번째 행을 비교해보면 NSP를 하지 않았을때 성능이 하락하는 모습을 볼 수 있는데, 이를 통해 NSP가 모델로 하여금 언어 능력을 향상하는데 도움을 준다는 사실을 알 수 있음.

두 번째는 양방향 학습의 효과로 두번째, 세번째 행을 비교해 보면 한방향 학습을 했을때 성능이 하락하는 모습을 볼 수 있음. 이를 통해 BERT가 제안하는 양방향 학습이 모델로 하여금 언어 능력을 향상하는데 도움을 준다는 사실을 알 수 있음.

6-3 Model Size 효과

Hyperparams				Dev Set Accuracy		
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. "LM (ppl)" is the masked LM perplexity of held-out training data.

전체적으로 모델 사이즈가 증가할 수록 성능이 좋아지는 모습을 볼 수 있음. 이를 통해 Model Size 증가는 큰 scale의 문제에서 뿐만 아니라 작은 Scale 문제에서도 성능 향상에 도움을 준다는 사실을 알 수 있음.

7-1 BERT vs GPT-1

- Architecture
 - GPT-1은 Transformer의 Decoder를 베이스로 모델을 구성. 이는 다음 단어를 예측해야 하는 GPT-1의 특성 때문.
 - BERT는 Transformer의 Encoder를 베이스로 모델을 구성. 이는 마찬가지로 Mask 단어와 두 문장간의 관계를 예측해야 하는 BERT의 특성 때문.
- 학습 방법
 - GPT-1은 다음 단어를 예측하며 Pretraining을 수행.
 - BERT는 두가지 방법을 같이 사용. 첫 번째는 Masking된 단어를 예측. 두 번째는 두 문장을 입력 받아 관계를 예측.
- 기능
 - 이는 위의 두 가지 차이점에 주목하면 이해할 수 있는데요.
 - GPT-1은 텍스트 생성 작업에 특화되어 있음. GPT-1은 애초에 다음 단어를 생성하도록 학습했기 때문.
 - BERT는 다양한 NLP Task에서 더 좋은 성능을 내도록 설계. 따라서 GPT-1 보다 대부분의 NLP 문제에서 좋은 성능을 보임
- 데이터 처리 방식
 - GPT-1은 한방향으로 데이터를 처리
 - BERT는 양방향으로 데이터를 해석

8 장단점

8-1. 장점

첫 번째 장점은 문맥 이해력임. BERT는 양방향으로 데이터를 처리해서 GPT 대비 문맥 정보를 더 잘 포착함. 이는 다양한 문제에서 GPT보다 높은 성능을 통해 확인할 수 있음

두 번째 장점은 Fine Tuning의 용이함. BERT는 대규모 데이터셋에서 Pretraining 한 뒤 작은 데이터셋에서 Fine Tuning을 수행하는데 이러한 방법을 통해 다양한 NLP 문제에서 높은 성능을 발휘할 수 있음

세 번째 장점은 다양한 작업 수행이 가능하다는 점. BERT는 Architecture 변경 없이 Fine Tuning 을 진행하는데 BERT의 범용성을 높여주는 하나의 요인이 됨.

8-2. 단점

첫 번째 단점은 계산 비용임. BERT는 GPT-1과 마찬가지로 기존 모델대비 엄청나게 많은 파라미터를 갖기 때문에 이로 인해 학습과 추론에 상당한 계산 비용이 요구됨.

두 번째 단점은 메모리 요구량으로 첫 번째 단점과 동일한 맥락에서 BERT를 사용하기 위해서는 엄청나게 많은 메모리가 요구됨.
세 번째 단점은 대량의 학습 데이터가 필요하다는 점. BERT의 장점은 강력한 Pretraining 성능인데 이를 위해서는 엄청난 양의 텍스트 데이터가 필요함. 이로 인해 BERT는 일반 개인이 쉽게 학습할 수 없다는 한계가 있음.